

Bài 1: Quản lý Dependency với Maven

Đề bài: Cho dự án `MathUtils` với tệp `pom.xml` cũ. Nhiệm vụ là nâng cấp cấu hình cũ (legacy) này để đáp ứng các yêu cầu sau:

- Thêm **Logback Classic 1.4.11** để thay thế các lệnh in thủ công (`System.out.println`) bằng structured logging (ghi log có cấu trúc).
- Thêm **Hibernate Core 6.2.0.Final** để hỗ trợ nhu cầu lưu trữ dữ liệu trong tương lai.
- Chuyển đổi dự án sang sử dụng **JUnit Jupiter 5.9.2** làm công cụ kiểm thử duy nhất.
- Giải quyết tất cả các xung đột và lỗi phát sinh từ những bổ sung này để đảm bảo quá trình build (xây dựng) thành công.
- Giải thích giải pháp.

Bài 2: Code Quality - Checkstyle

Đề bài: Một dự án `BankSystem` cũ chứa nhiều lỗi thiết kế và phong cách code không chuẩn. Mục tiêu là áp dụng các tiêu chuẩn chất lượng code chuyên nghiệp và khả năng quan sát (observability) bằng cách thực hiện các bước sau:

- Tích hợp **Maven Checkstyle Plugin** vào quy trình build của dự án.
- Chọn và áp dụng một tiêu chuẩn code (có thể dùng **Google Java Style** hoặc **Sun Code Conventions**) để xác định tất cả các vi phạm về định dạng và đặt tên.
- Tái cấu trúc (Refactor) mã nguồn để giải quyết tất cả các lỗi được tìm thấy và vượt qua goal `checkstyle:check` thành công.
- Theo dõi trạng thái hoạt động của hệ thống thông qua logging (sử dụng **SLF4J** hoặc **Logback**). Giải thích lý do chọn các cấp độ log (logging levels) và các điểm dữ liệu bạn chọn để ghi lại.

Bài 3: CI/CD automation

Đề bài: Chọn một hàm hoặc hệ thống đã triển khai trước đó và tổ chức lại thành một dự án Maven có tích hợp logging SLF4J/Logback và Unit Tests. Nhiệm vụ là định nghĩa một quy trình CI để tự động hóa việc xác minh build:

- Viết một workflow **GitHub Actions** được kích hoạt (trigger) khi có sự kiện `push` và `pull_request`.
- Cấu hình workflow để thực thi vòng đời build của Maven (Maven build lifecycle), đảm bảo các pha `test` và `package` chạy thành công.
- Triển khai action `upload-artifact` để lưu giữ và xác minh tệp `.jar` được tạo ra sau khi build hoàn tất.
- Kiểm thử sự tự động hóa bằng cách cố tình gây ra lỗi build và chứng minh khả năng debug vấn đề bằng cách sử dụng execution logs do GitHub Actions cung cấp.

Bài 4: Kiểm thử đa hệ điều hành với Matrix Strategy

Đề bài: Vấn đề “Nó chạy được trên máy tôi” (It works on my machine) thường xảy ra do sự khác biệt về hệ điều hành. GitHub Actions cung cấp chiến lược Matrix để giúp giải quyết vấn đề này. Yêu cầu:

- Cập nhật tệp cấu hình workflow của bạn để áp dụng **Matrix Strategy**, cho phép các bài test chạy đồng thời trên nhiều hệ điều hành: `ubuntu-latest`, `windows-latest`, và `macos-latest`.
- Cố tình tạo một Unit Test xử lý đường dẫn tệp tin sử dụng định dạng cứng của Windows hoặc Linux (ví dụ: dùng dấu gạch chéo cứng `\` hoặc `/`). Chạy pipeline để quan sát lỗi xảy ra trên hệ điều hành không tương thích.
- Tái cấu trúc (Refactor) mã nguồn sử dụng `File.separator` hoặc API `java.nio.file.Path` để đảm bảo bài test chạy thành công trên mọi môi trường trong matrix.

Bài 5: Test Coverage & Quality Enforcement (JaCoCo)

Đề bài: Để đảm bảo độ tin cậy của mã nguồn, chỉ viết unit test là chưa đủ; bạn phải đo lường xem bao nhiêu phần code của bạn thực sự được kiểm thử (Code Coverage).

- Tích hợp `jacoco-maven-plugin` vào `pom.xml` để tạo báo cáo về độ bao phủ code.
- Cấu hình một quy tắc nghiêm ngặt (strict rule) trong plugin để tự động làm **fail** bản build nếu độ bao phủ code giảm xuống dưới **80%**.
- Đảm bảo bước kiểm tra này được thực thi trong workflow GitHub Actions của bạn (ví dụ: trong pha `mvn verify`).

- Sử dụng action `upload-artifact` để trích xuất và lưu trữ báo cáo bao phủ (tìm thấy tại `target/site/jacoco/index.html`) sau khi build xong để có thể xem lại sau.

Bài 6: CI/CD Pipeline Optimization & Caching

Đề bài: Khi dự án phát triển, việc tải xuống các Maven dependency từ đầu cho mỗi lần chạy CI tiêu tốn nhiều thời gian và làm chậm chu kỳ phát triển.

- Cấu hình **dependency caching** trong workflow GitHub Actions bằng cách thêm tham số `cache: 'maven'` vào action `setup-java`.
- Thực hiện 2 lần push code liên tiếp lên GitHub repository. Ghi lại và so sánh thời gian thực thi workflow trước và sau khi áp dụng caching.
- Kiểm tra và phân tích log thực thi của GitHub Actions để chứng minh rằng các dependency đã được lấy thành công từ cache thay vì phải tải lại từ Maven Central.

Bài 7: Automated Code Review via Pull Request

Đề bài: Biến CI pipeline của bạn thành một “người review” tự động để cung cấp phản hồi ngay lập tức trong quá trình phát triển. **Yêu cầu:**

- **Trigger:** Cập nhật workflow GitHub Actions để tự động trigger khi có sự kiện `pull_request` hướng vào nhánh `main`.
- **Inline Feedback (Phản hồi trực tiếp):** Tích hợp một GitHub Action (ví dụ: `dbelyaev/action-checkstyle`) để tự động đăng “bình luận” (comments) trực tiếp vào các dòng code vi phạm tiêu chuẩn dự án.
- **Branch Protection (Bảo vệ nhánh):** Cấu hình **Branch Protection Rules** để vô hiệu hóa nút Merge nếu phát hiện bất kỳ lỗi định dạng (formatting errors) hoặc lỗi test nào.
- **Verification (Kiểm chứng):** Tạo một Pull Request với một lỗi Checkstyle cố ý. Xác nhận rằng Bot tự động bình luận vào PR và nút Merge vẫn bị khóa cho đến khi lỗi được sửa.

Bài 8: Đóng gói sản phẩm thực thi

Nhiệm vụ: Trong môi trường chuyên nghiệp, phần mềm phải được đóng gói để có thể chạy độc lập mà không cần đến IDE. Bạn được yêu cầu cấu hình vòng

đòi Maven để tạo ra một tệp thực thi hoàn chỉnh.

- Yêu cầu:
 - Cấu hình plugin maven-jar-plugin trong tệp pom.xml bằng cách chỉ định lớp chính (Main-Class) trong phần manifest.
 - Thực thi lệnh mvn clean package để tạo tệp .jar trong thư mục target.
 - Xác minh: Chứng minh ứng dụng có thể khởi chạy thông qua terminal bằng lệnh java -jar target/*.jar. Giải thích ý nghĩa của thư mục target và pha package trong Maven.

Bài 9: Triển khai Logging chuyên nghiệp

Nhiệm vụ: Thay thế tất cả các thói quen kiểm thử lỗi "nghề nghiệp" bằng một khung làm việc logging có cấu trúc để đảm bảo khả năng truy vết hệ thống, như đã thảo luận trong bài giảng.

- **Yêu cầu:**
 - Tối ưu hóa mã nguồn bằng cách loại bỏ tất cả các câu lệnh `System.out.println()`.
 - Triển khai **SLF4J** cùng với **Logback** bằng cách sử dụng các mức độ log phù hợp: **INFO** để theo dõi các mốc quan trọng và **ERROR** để xử lý ngoại lệ.
 - Sử dụng **Parameterized Logging** (dấu giữ chỗ `{}`) thay vì cộng chuỗi để tối ưu hóa hiệu suất.
 - Cấu hình một **FileAppender** trong tệp **logback.xml** để lưu trữ log vào một tệp vật lý.
- **Xác minh:** Chạy lệnh **mvn test** và xác nhận rằng các dòng log được định dạng chính xác (bao gồm thời gian và mức độ log) trong cả console và tệp log được tạo ra.

Bài 10: The broken pipeline

Một dự án Maven đã được thiết lập sẵn nhưng pipeline CI liên tục báo đỏ. Nhiệm vụ của bạn là tìm và sửa tất cả lỗi mà không được sửa mò - phải đọc log để hiểu nguyên nhân trước khi sửa.

Dự án được cung cấp như sau:

src/main/java/com/lab/ShippingCalculator.java

```

package com.lab;

public class ShippingCalculator {

    public double calculate(double weight, String type) {
        if (weight <= 0) {
            throw new IllegalArgumentException("Weight must be
positive");
        }
        if (type.equals("EXPRESS")) return weight * 5000 +
20000;
        if (type.equals("STANDARD")) return weight * 3000;
        throw new IllegalArgumentException("Unknown type: " +
type);
    }
}

```

src/test/java/com/lab/ShippingCalculatorTest.java

```

package com.lab;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ShippingCalculatorTest {

    ShippingCalculator calc = new ShippingCalculator();

    @Test
    void testStandard() {
        assertEquals(15000.0, calc.calculate(5, "STANDARD"));
    }

    @Test
    void testExpress() {
        assertEquals(45000.0, calc.calculate(5, "EXPRESS"));
    }
}

```

```

    }

    @Test
    void testInvalidWeight() {
        assertThrows(IllegalArgumentException.class,
            () -> calc.calculate(-1, "STANDARD"));
    }
}

```

pom.xml

```

<project>
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.lab</groupId>
    <artifactId>shipping-app</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <dependencies>
        <dependency>
            <groupId>ch.qos.logback</groupId>
            <artifactId>logback-classic</artifactId>
            <version>9.9.9</version>
        </dependency>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.9.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

```

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>2.12.4</version>
    </plugin>
  </plugins>
</build>
</project>
```

.github/workflows/ci.yml

name: Java CI

on: [push, pull_request]

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Set up JDK 17
uses: actions/setup-java@v3
with:
 java-version: '17'
 distribution: 'temurin'
- name: Build with Maven
run: mvn package

Yêu cầu:

1. Tạo repository trên GitHub, đẩy toàn bộ dự án lên và quan sát pipeline chạy.
2. Với mỗi lỗi tìm được:
 - Chỉ ra lỗi nằm ở file nào, dòng nào.
 - Sao chép đoạn log liên quan từ GitHub Actions làm bằng chứng.
 - Giải thích nguyên nhân kỹ thuật của lỗi.

- Sửa và push lại - pipeline phải xanh hoàn toàn trước khi chuyển sang lỗi tiếp theo.
3. Sau khi pipeline xanh, cố tình tạo thêm một lỗi thứ 4 tự nghĩ ra, push lên, chụp ảnh pipeline đỏ, sửa lại và giải thích. Lỗi tự nghĩ không được trùng với 3 lỗi đã có.

- 1.